Experiment 'p_6_4_sum_without_smallest_seq' Results

June 4, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_6_4_sum_without_smallest_seq
Natural language statement: Write a method that computes the sum of an array of values, except for the smallest one, in a single loop. In the loop, update the sum and the smallest value. After the loop, return the difference.
Method signature: p_6_4_sum_without_smallest_seq(inputs: seq<int>) returns (sum: int)

Ensures

- `sum == sum_array(inputs) - find_smallest(inputs)`

Requires

- `|inputs| >= 1`

Functional Code Given

```
function find_smallest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> ret <= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_smallest := find_smallest(inputs[1..]);
    var ret: int := if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    assert inputs == [inputs[0]] + inputs[1..];
    assert ret == if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    ret
}
```

```
function sum_array(inputs: seq<int>): int
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that computes the sum of an array of values, except for the smallest one,
    ↪ in a single loop. In the loop, update the sum and the smallest value. After the
    ↪ loop, return the difference.

The signature should be:

p_6_4_sum_without_smallest_seq(inputs: seq<int>) returns (sum: int)

The method should respect the following contract:

requires |inputs| >= 1, ensures sum == sum_array(inputs) - find_smallest(inputs)

The contract uses the following dafny code:

function find_smallest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> ret <= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_smallest := find_smallest(inputs[1..]);
    var ret: int := if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    assert inputs == [inputs[0]] + inputs[1..];
    assert ret == if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    ret
}

function sum_array(inputs: seq<int>): int
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn1VHBqfhXOwY3jUcSdq5XDfBGX1c
Model: gpt-5.5-2026-04-23
Created at: 1780576223
Finish reason: stop
Usage: 760 tokens in, and 6749 tokens out

```
{
  var i: int := |inputs| - 1;
  assert 0 <= i < |inputs|;

  var runningSum: int := inputs[i];
  var smallest: int := inputs[i];

  assert |inputs[i..]| == 1;
  assert (inputs[i..])[0] == inputs[i];
  assert sum_array(inputs[i..]) == inputs[i];
  assert find_smallest(inputs[i..]) == inputs[i];

  while 0 < i
    invariant 0 <= i < |inputs|
    invariant runningSum == sum_array(inputs[i..])
    invariant smallest == find_smallest(inputs[i..])
    decreases i
  {
    var oldI: int := i;
    i := i - 1;

    assert oldI == i + 1;
    assert 0 <= i;
    assert i + 1 < |inputs|;
    assert runningSum == sum_array(inputs[i + 1..]);
    assert smallest == find_smallest(inputs[i + 1..]);

    assert |inputs[i..]| >= 2;
    assert (inputs[i..])[0] == inputs[i];
    assert (inputs[i..])[1..] == inputs[i + 1..];

    assert sum_array(inputs[i..]) == (inputs[i..])[0] + sum_array((inputs[i..])[1..]);
    assert sum_array(inputs[i..]) == inputs[i] + sum_array(inputs[i + 1..]);

    assert find_smallest(inputs[i..]) ==
      (if (inputs[i..])[0] <= find_smallest((inputs[i..])[1..]) then (inputs[i..])[0] else
          ↪  find_smallest((inputs[i..])[1..]));
    assert find_smallest(inputs[i..]) ==
      (if inputs[i] <= find_smallest(inputs[i + 1..]) then inputs[i] else find_smallest(
          ↪ inputs[i + 1..]));

    runningSum := inputs[i] + runningSum;
    assert runningSum == inputs[i] + sum_array(inputs[i + 1..]);
```

```
    assert runningSum == sum_array(inputs[i..]);

    if inputs[i] <= smallest {
      assert inputs[i] <= find_smallest(inputs[i + 1..]);
      smallest := inputs[i];
    } else {
      assert !(inputs[i] <= find_smallest(inputs[i + 1..]));
    }
    assert smallest ==
      (if inputs[i] <= find_smallest(inputs[i + 1..]) then inputs[i] else find_smallest(
          ↪ inputs[i + 1..]));
    assert smallest == find_smallest(inputs[i..]);
  }

  assert i == 0;
  assert inputs[i..] == inputs;
  assert runningSum == sum_array(inputs);
  assert smallest == find_smallest(inputs);
  sum := runningSum - smallest;
}
```

Final Program

```
function find_smallest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> ret <= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_smallest := find_smallest(inputs[1..]);
    var ret: int := if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    assert inputs == [inputs[0]] + inputs[1..];
    assert ret == if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    ret
}

function sum_array(inputs: seq<int>): int
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}

method p_6_4_sum_without_smallest_seq(inputs: seq<int>) returns (sum: int)
       requires |inputs| >= 1
       ensures sum == sum_array(inputs) - find_smallest(inputs)
{
  var i: int := |inputs| - 1;
  assert 0 <= i < |inputs|;
```

```
var runningSum: int := inputs[i];
var smallest: int := inputs[i];

assert |inputs[i..]| == 1;
assert (inputs[i..])[0] == inputs[i];
assert sum_array(inputs[i..]) == inputs[i];
assert find_smallest(inputs[i..]) == inputs[i];

while 0 < i
  invariant 0 <= i < |inputs|
  invariant runningSum == sum_array(inputs[i..])
  invariant smallest == find_smallest(inputs[i..])
  decreases i
{
  var oldI: int := i;
  i := i - 1;

  assert oldI == i + 1;
  assert 0 <= i;
  assert i + 1 < |inputs|;
  assert runningSum == sum_array(inputs[i + 1..]);
  assert smallest == find_smallest(inputs[i + 1..]);

  assert |inputs[i..]| >= 2;
  assert (inputs[i..])[0] == inputs[i];
  assert (inputs[i..])[1..] == inputs[i + 1..];

  assert sum_array(inputs[i..]) == (inputs[i..])[0] + sum_array((inputs[i..])[1..]);
  assert sum_array(inputs[i..]) == inputs[i] + sum_array(inputs[i + 1..]);

  assert find_smallest(inputs[i..]) ==
    (if (inputs[i..])[0] <= find_smallest((inputs[i..])[1..]) then (inputs[i..])[0] else
        ↪  find_smallest((inputs[i..])[1..]));
  assert find_smallest(inputs[i..]) ==
    (if inputs[i] <= find_smallest(inputs[i + 1..]) then inputs[i] else find_smallest(
        ↪ inputs[i + 1..]));

  runningSum := inputs[i] + runningSum;
  assert runningSum == inputs[i] + sum_array(inputs[i + 1..]);
  assert runningSum == sum_array(inputs[i..]);

  if inputs[i] <= smallest {
    assert inputs[i] <= find_smallest(inputs[i + 1..]);
    smallest := inputs[i];
  } else {
    assert !(inputs[i] <= find_smallest(inputs[i + 1..]));
  }
  assert smallest ==
    (if inputs[i] <= find_smallest(inputs[i + 1..]) then inputs[i] else find_smallest(
        ↪ inputs[i + 1..]));
  assert smallest == find_smallest(inputs[i..]);
```

```
  }

  assert i == 0;
  assert inputs[i..] == inputs;
  assert runningSum == sum_array(inputs);
  assert smallest == find_smallest(inputs);
  sum := runningSum - smallest;
}
```

Total Token Usage

Input tokens: 760
Output tokens: 6749
Reasoning tokens: 5997
Sum of 'total tokens': 7509

Experiment Timings

Overall Experiment started at 1780576222306, ended at 1780576379202, lasting 156896ms (156.90 seconds)
Iteration #1 started at 1780576222311, ended at 1780576379202, lasting 156891ms (156.89 seconds)